manages where and how images are stored on your filesystem. We list the images on our system with the following command:

```
$ docker images
REPOSITORY     TAG         IMAGE ID        CREATED         SIZE
<none>         <none>      a1df0eddba18    1 second ago    1.01GB
ruby           2.6         f28a9e1d0449    6 days ago      868MB
```

The first entry is the custom image that we just built—its image ID matches the one specified at the end of the `docker build` command.

Running a Rails Server with Our Image

Now that we've created our own tailor-made image, we should be able to start up a Rails server to run our app.

Let's try doing that now.

We can refer to our image by its ID. However, image IDs are long and hard to remember, so normally you would assign a meaningful name to an image. We'll see how to do that later in Naming and Versioning Our Image, on page 31. For now, though, referring to our image by its ID is good enough to let us start using the image. Let's crack on and do just that.

Image ID in hand, we can start our Rails app inside a container based on our custom image with the following command. Let's run it now:

```
$ docker run -p 3000:3000 a1df0eddba18 \
    bin/rails s -b 0.0.0.0
```

Aside from the new `-p 3000:3000` option, which we'll cover shortly in Reaching the App: Publishing Ports, on page 26, this is a plain old `docker run` command. It says, "Start a container based on our custom image (`a1df0eddba18`), and run `bin/rails s -b 0.0.0.0` inside it." If you haven't seen the `-b` option before, we'll explain why this is needed in Binding the Rails Server to IP Addresses, on page 28.

We should see that Rails starts up correctly:

```
=> Booting Puma
=> Rails 5.2.2 application starting in development
=> Run `rails server -h` for more startup options
Puma starting in single mode...
* Version 3.12.0 (ruby 2.6.0-p0), codename: Llamas in Pajamas
* Min threads: 5, max threads: 5
* Environment: development
* Listening on tcp://0.0.0.0:3000
Use Ctrl-C to stop
```

It does! So far so good.